서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 후 보고서

STM32 UART 통신 및 FPGA 서보모터·SRAM·DAC 실습

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 11월 01일
대상	STM32 과제5 · STM32 과제6 · STM32 과제7 · FPGA 서보모터 · SRAM · DAC 실습
검증환경	Windows 11 / VS Code / STM32 CMake / VaporView / AMD Vivado 2026.1 / XSim / Serial Monitor

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab6

제출 태그: lab6-post-v1

커밋 해시: a0760819fa1a283d70cab8113514d0af3a75693b

1. 실험 목적

이번 실험에서는 PC에서 입력한 문자를 STM32가 수신하고, 이를 FPGA에 전달하는 UART 통신 경로를 확인한다. 과제 5에서는 수신 문자를 문자와 16진수 바이트 값으로 출력한다. 과제 6에서는 STM32가 받은 바이트를 FPGA로 보내고, FPGA가 다시 PC에 출력하는지 확인한다. 과제 7에서는 받은 문자를 명령으로 해석하여 FPGA LED를 제어한다.

FPGA 개별 실습에서는 서보모터에 전달할 PWM 펄스의 폭을 바꾸고, SRAM의 쓰기·읽기와 양방향 데이터 버스 제어를 확인하며, DAC에 전달하는 8비트 데이터를 증가시켜 보드 출력을 관찰한다. 각 설계를 Verilog 시뮬레이션, Vivado 구현 결과, 실제 보드 시연과 비교한다.

2. 관련 이론

SRAM 읽기·쓰기와 양방향 데이터 버스

SRAM(Static Random Access Memory)은 주소로 지정한 위치에 데이터를 저장하고, 같은 주소를 다시 지정해 그 값을 읽는 메모리다. 데이터를 유지하기 위해 DRAM과 같은 주기적인 리프레시가 필요하지 않지만, 전원이 끊기면 저장된 데이터는 유지되지 않는다. SRAM에는 접근할 위치를 지정하는 주소선, 값을 주고받는 데이터선, 읽기와 쓰기를 선택하는 제어 신호가 있다.

데이터선은 쓰기와 읽기에 함께 사용될 수 있다. 쓰기 때에는 제어 회로가 주소와 데이터를 내보내고 쓰기 신호를 활성화한다. 읽기 때에는 SRAM이 지정한 주소의 값을 데이터선에 출력한다. 따라서 읽는 동안 제어 회로는 자신의 데이터 출력을 해제해야 한다. 이때의 고임피던스 상태 Z는 데이터선에 0이나 1을 출력하지 않는 상태를 뜻한다. 두 회로가 같은 데이터선을 동시에 구동하지 않도록 하는 것이 양방향 버스 제어의 핵심이다.[1]

3. 실험 환경 및 검증 방법

사용 장비와 연결

항목	설정 및 용도
STM32	NUCLEO-G474RE, PC 입력 수신 및 UART3 송신
FPGA	Combo II-DLD S75, UART Echo 및 LED 제어 실습
PC Serial Monitor	문자 입출력 확인, UART 9600 BPS·8N1
MCU-FPGA 연결	STM32 PC 10(TX) – FPGA 확장 GPIO UART RX, 양쪽 GND 공통
FPGA의 PC 출력	FPGA UART TX – 보드 USB-UART – PC
FPGA 구현	Vivado 합성 · 구현 · 비트스트림 생성 후 보드 시연
결과 관리	Git 및 GitHub

검증 순서

과제 6·7에서는 PC에서 STM32의 가상 COM 포트로 문자를 보내고, STM32가 UART3로 해당 바이트를 FPGA에 전달하였다. FPGA 출력은 별도의 USB-UART 포트에서 확인하였다. STM32 쪽에 표시되는 TX result: 0은 송신 함수가 정상 종료했다는 뜻이며, FPGA 수신 성공 여부는 FPGA 쪽 출력으로 따로 확인하였다.

실험별 설계 및 증빙 파일

항목	설정
과제6·과제7	소스·테스트벤치·핀 설정
실습 1 서보모터	소스·테스트벤치·핀 설정
실습 2 SRAM	소스·테스트벤치·핀 설정
실습 3 DAC	소스·테스트벤치·핀 설정

4. STM32 과제5: 수신 문자와 바이트 값 출력

설계 및 예상 동작

PC 시리얼 모니터에서 문자 한 개를 전송하면 STM32가 이를 받아 문자 자체와 해당 바이트의 16진수 값을 출력하도록 하였다. 예를 들어 문자 A의 값은 0x41, 소문자 a의 값은 0x61로 나타나야 한다. 이 단계에서는 FPGA 연결 없이 PC와 STM32의 입출력을 먼저 확인한다.

코드 동작

polling_routine()은 getchar()로 입력한 문자를 읽는다. 반환값은 입력 종료 상태인 EOF도 표현할 수 있도록 int로 받고, 실제 문자가 들어온 경우 uint8_t로 변환하여 출력한다. printf()의 %c는 문자를, 0x%02X는 바이트 값을 최소 두 자리의 대문자 16진수로 표시한다.

빌드 및 보드 결과

과제 5 코드를 빌드한 뒤 STM32의 가상 COM 포트를 Serial Monitor에 연결하였다. 저장한 출력 화면에는 Z → 0x5A, A → 0x41, a → 0x61이 나타난다. 문자 A와 a가 각각 다른 바이트 값으로 출력되어 예상한 문자 처리 결과와 일치하였다. 전송 설정에 줄바꿈을 포함하면 줄바꿈 문자도 별도로 읽을 수 있으므로, 문자 하나를 비교할 때에는 전송 설정을 확인하였다.

```
---- Sent utf8 encoded message: "Z" ----
Character: Z, Byte: 0x5A
---- Sent utf8 encoded message: "A" ----
Character: A, Byte: 0x41
---- Sent utf8 encoded message: "a" ----
Character: a, Byte: 0x61
```

그림 1: 과제5 문자와 16진수 바이트 출력

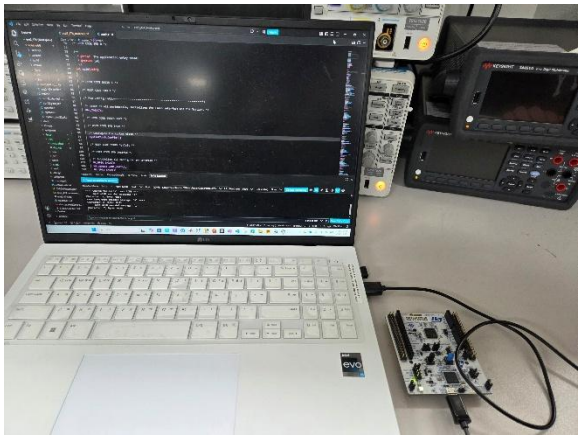


그림 2: STM32와 PC 시리얼 모니터 시연

링크: 과제5 시연 영상

5. STM32 과제6: UART 문자 전달

설계 및 예상 동작

PC에서 보낸 문자를 STM32가 받은 뒤 UART3로 FPGA에 전달한다. FPGA는 받은 바이트를 UART Echo 회로로 다시 송신하므로, PC에서는 STM32에 입력한 문자와 FPGA 쪽 포트에 나타난 문자를 비교할 수 있어야 한다. 이때 MCU의 송신 함수가 정상 종료했다는 결과와 FPGA가 실제로 문자를 받았다는 결과는 구별해서 확인한다.

코드 동작

STM32는 getchar()로 얻은 입력을 uint8_t 한 바이트로 변환하고 HAL_UART_Transmit(&huart3, &data, 1U, 100U)로 송신한다. 반환값 0은 이 함수의 정상 종료를 나타낸다. FPGA의 lab3_uart_echo는 UART 입력을 받아 같은 바이트를 송신한다. 모듈 이름에 lab3가 남아 있는 것은 lab3 실험 소스를 사용하였음을 나타낸다.

Vivado 실험사진

```
run all
LAB3_UART_ECHO_PASS checks=3
$finish called at time : 9910 ns : File "D:/Engineering/Verilog_Lab/ece2-lab6/fpga/uart/sim/tb_uart_echo.sv" Line 108
```

그림 3: 과제6 Vivado 시뮬레이션 PASS

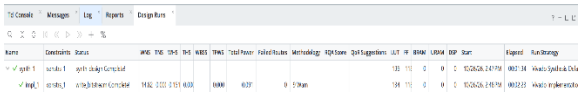


그림 4: 과제6 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	134	48000	0.28
FF	118	96000	0.12
IO	12	338	3.55

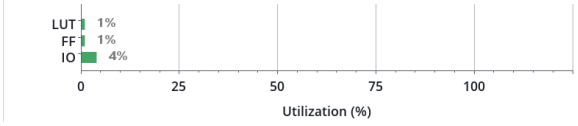


그림 5: 과제6 FPGA 자원 사용량

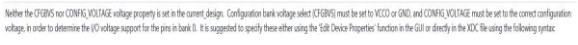


그림 6: 과제6 DRC 검사 결과

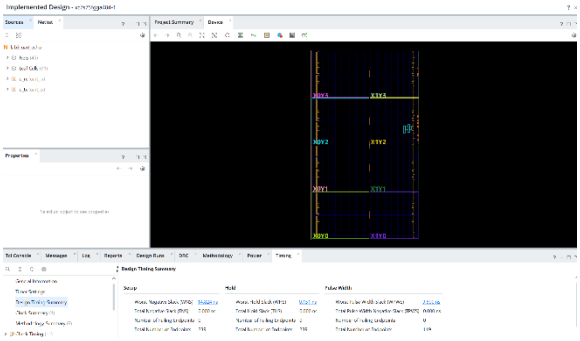


그림 7: 과제6 타이밍 분석 결과

```
PS D:\Engineering\Vivado_Work\Lab6> Get-FileHash -Algorithm SHA256 -LiteralPath 'D:\Engineering\Vivado_Work\Lab6\lab6_uart_echo\lab6_uart_echo.runs\impl_1\lab3_uart_echo.bit'

Algorithm       Hash                                                             Path
---------       ----                                                             ----
SHA256          CDE9A8A0C509D181D6F7AE70C1407B9FE666F2E2DE82B958605625A85FA4376  D:\Engineering\Vivado_Work\La...
```

그림 8: 과제6 Bit Hash 확인

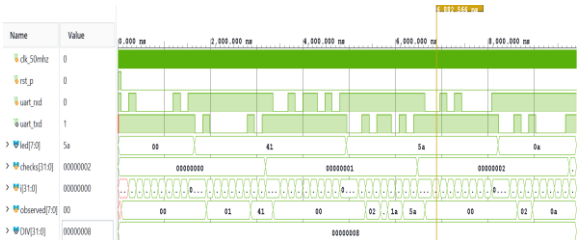


그림 9: 과제6 UART 수신 및 ECHO 송신 파형

Vivado 시뮬레이션 및 파형 비교

과제 6의 Vivado 시뮬레이션은 PASS로 종료되었다. 그림 9에서는 UART 수신선에 바이트가 들어온 뒤 송신선에 Echo 데이터가 출력된다. 파형에 표시된 수신 데이터에는 0x41, 0x5A, 0x0A가 포함된다. 0x0A는 줄바꿈 문자이므로, 문자 입력과 함께 전송된 경우 별도 바이트로 처리된다.

FPGA 보드 시연

STM32 코드를 빌드하여 실행하고 PC에서 입력한 문자를

STM32 UART3를 통해 FPGA로 전달하였다. 그림 10의 왼쪽 모니터에는 STM32가 받은 문자와 TX result: 0이 기록되어 있고, 오른쪽 모니터에는 FPGA가 출력한 문자 기록이 남아 있다. TX result: 0은 MCU의 송신 함수가 정상 종료했다는 결과이므로, FPGA 수신 여부는 오른쪽 출력과 비교하였다.

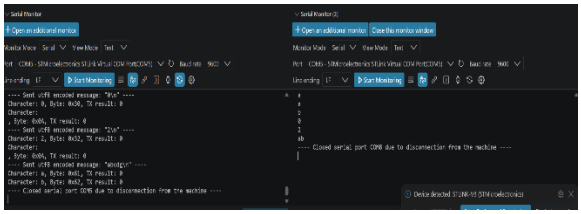


그림 10: 과제6 STM32 송신 및 FPGA Echo 출력 기록

링크: [과제6 보드 시연 영상]

6. STM32 과제7: UART 명령으로 LED 제어

설계 및 예상 동작

과제 6과 동일하게 PC에서 입력한 문자를 STM32가 FPGA로 전달한다. FPGA는 수신한 문자를 명령으로 해석하여 지정 LED의 상태를 바꾼다. 문자 0은 LED를 끄고, 1은 켜며, 2는 현재 상태를 반전한다. 그 밖의 문자를 받으면 이전 상태를 유지한다.

PC 입력	수신 바이트	LED 동작
0	0x30	끄기
1	0x31	켜기
2	0x32	현재 상태 반전

코드 및 회로 동작

STM32는 과제 6과 같이 입력 문자 한 바이트를 UART3로 전달한다. FPGA의 lab6_uart_led에서는 UART 수신값을 판별해 led_state를 변경한다. led[0]에는 이 상태를 출력하고 led[7:1]은 0으로 둔다. 수신 시 송신기가 준비된 경우에는 받은 바이트를 FPGA의 UART 출력으로도 보낸다.

Vivado 실험사진



그림 11: 과제7 Vivado 시뮬레이션 PASS

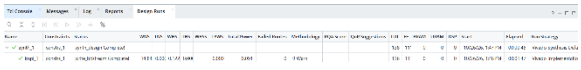


그림 12: 과제7 합성 및 구현 완료 결과

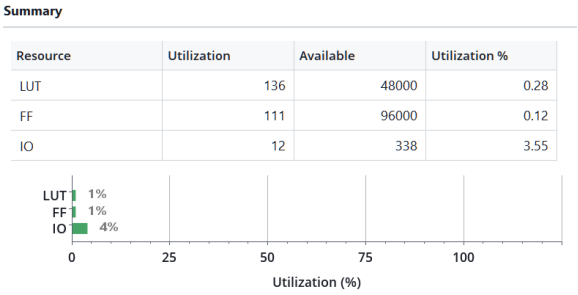


그림 13: 과제7 FPGA 자원 사용량

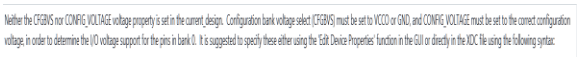


그림 14: 과제7 DRC 검사 결과

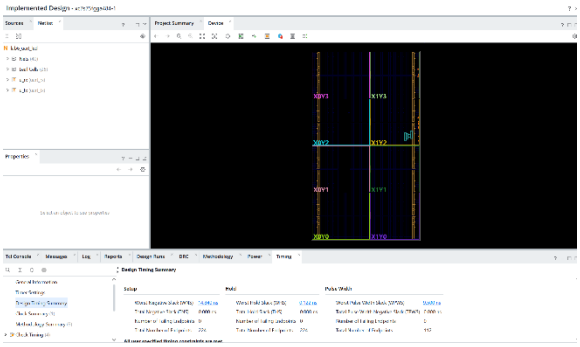


그림 15: 과제7 타이밍 분석 결과



그림 16: 과제7 Bit Hash 확인

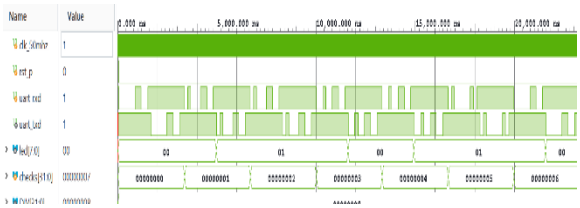


그림 17: 과제7 명령에 따른 LED 출력 파형

Vivado 시뮬레이션 및 파형 비교

Vivado 시뮬레이션은 PASS로 종료되었다. 그림 17에서 led[7:0]은 명령 입력에 따라 00 → 01 → 00 → 01 → 00으로 바뀐다. 지정한 led[0]이 켜짐과 꺼짐을 반복하고 나머지 비트는 0으로 유지되는 결과다. 문자 2에 대해서는 현재 상태를 기준으로 출력이 반전되는지 확인하였다.

FPGA 보드 시연

STM32와 FPGA의 송수신선 및 공통 GND를 연결하여 문자 명령을 전달하였다. 그림 18에는 두 보드의 연결과 LED가 켜진 한 시점이 나타난다. 정지 사진 한 장으로 0,

1, 2 명령을 보냈을 때의 연속적인 상태 변화를 모두 나타낼 수 없으므로 시연 영상을 함께 제시한다.

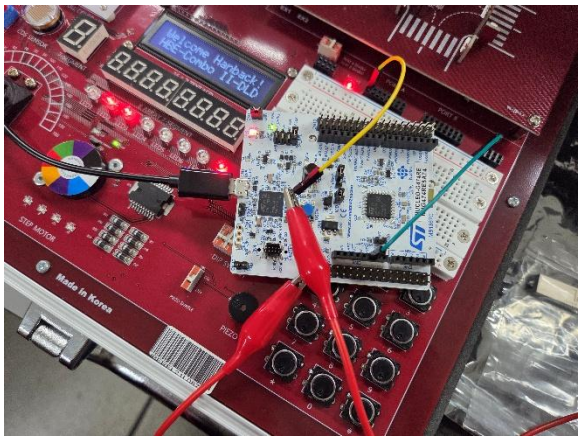


그림 18: 과제7 STM32-FPGA 연결 및 LED 시연

링크: [과제7 보드 시연 영상]

7. FPGA 실습1: 서보모터 제어

회로 구성

servo_position은 버튼 입력에 따라 서보 제어용 펄스 폭 설정값을 변경한다. 기본값은 1500 μs이고, 한 번의 입력마다 100 μs씩 바뀌며 1000~2000 μs 범위를 벗어나지 않는다. servo_pwm은 이 값을 이용해 20 ms 주기의 PWM 신호를 생성한다. 보드용 최상위 모듈 servo_board에서는 버튼 입력을 한 번의 펄스로 처리한 뒤 제어 회로에 전달한다.

Vivado 실험사진

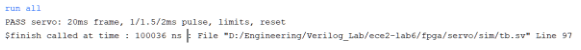


그림 19: 실습1 Vivado 시뮬레이션 PASS



그림 20: 실습1 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	138	48000	0.29
FF	127	96000	0.13
IO	5	338	1.48

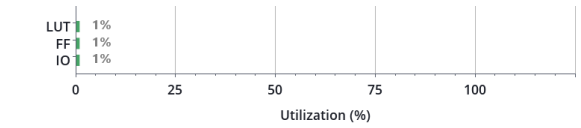


그림 21: 실습1 FPGA 자원 사용량



그림 22: 실습1 DRC 검사 결과

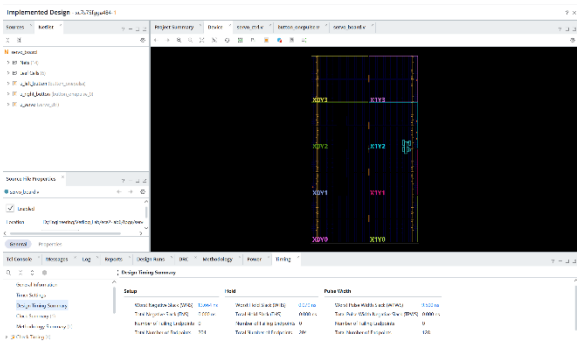


그림 23: 실습1 타이밍 분석 결과



그림 24: 실습1 Bit Hash 확인

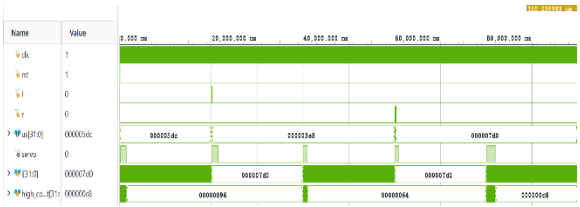


그림 25: 실습1 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

그림 25에서 펄스 폭 설정값은 0x05DC → 0x03E8 → 0x07D0으로 바뀐다. 이를 10진수로 나타내면 1500 → 1000 → 2000 μs이다. servo 출력의 HIGH 구간도 설정값에 맞춰 변하므로, 입력에 따라 PWM 펄스 폭이 변경되는 것을 확인할 수 있다.

FPGA 보드 시연

보드에서 4번 버튼을 누르면 서보모터가 왼쪽으로 이동하고, 6번 버튼을 누르면 오른쪽으로 이동하는 것을 확인하였다. 5번 리셋 버튼을 누르면 펄스 폭 설정값이 초기값인 1500 μs로 돌아가면서 서보모터도 중간 위치로 복귀하였다. 그림 26은 서보모터를 연결한 보드의 시연 사진이며, 버튼 4·5·6을 차례로 눌렀을 때의 위치 변화는 시연 영상에 기록하였다.

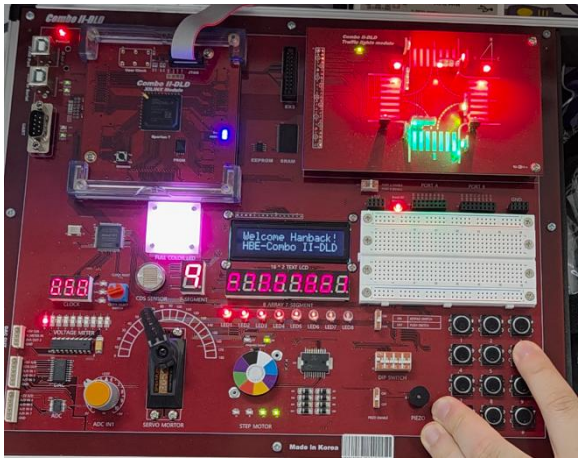


그림 26: 실습1 FPGA 보드 시연 사진

링크: FPGA 실습1 보드 시연 영상

8. FPGA 실습2: SRAM 읽기 및 쓰기

회로 구성

sram_bus_control은 읽기와 쓰기 요청에 따라 SRAM 제어 신호 및 데이터 버스 방향을 결정한다. sram은 입력된 4비트 주소를 메모리 주소선의 하위 비트에 연결한다. 쓰기 때에는 입력 데이터의 하위 4비트를 외부 데이터 버스에 출력하고, 읽기 때에는 FPGA의 버스 출력을 고임피던스 상태 Z로 놓는다. 읽은 하위 4비트는 LED 출력으로 확인하도록 연결하였다.

Vivado 실험사진

```
INFO: [USF-XSim-96] XSim completed. Design snapshot 'tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000ns
launch_simulation: Time (s): cpu = 00:00:10 ; elapsed = 00:00:10 . Memory (MB): peak = 1752.508 ; gain = 9.773
```

그림 27: 실습2 Vivado 시뮬레이션 PASS

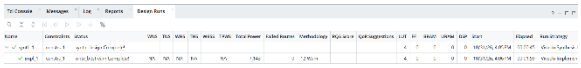


그림 28: 실습2 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	4	48000	0.01
IO	55	338	16.27

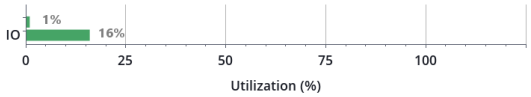


그림 29: 실습2 FPGA 자원 사용량



그림 30: 실습2 DRC 검사 결과

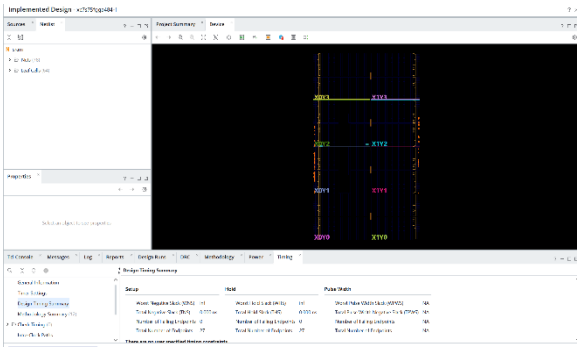


그림 31: 실습2 타이밍 분석 결과



그림 32: 실습2 Bit Hash 확인

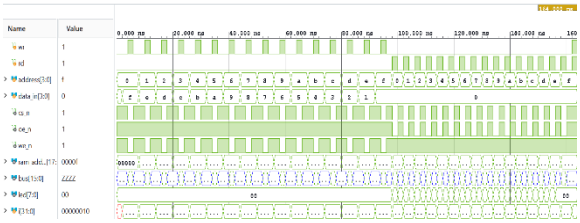


그림 33: 실습2 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

그림 33의 앞부분에서는 쓰기 요청에 따라 주소가 0부터 F까지 바뀌고 입력 데이터가 F부터 0까지 변한다. 뒤의 읽기 구간에서는 주소 0부터 F까지 다시 접근하며, 제어 신호와 데이터 버스의 구동 방향이 쓰기 구간과 달라진다. 이를 통해 주소별 쓰기·읽기 순서와 읽기 중 FPGA의 버스 해제를 확인하였다.

FPGA 보드 시연

보드의 왼쪽 하위 스위치 4개로 저장할 데이터 값을 설정하고 2번(K4) 버튼을 눌러 SRAM에 기록하였다. 이어서 3번 버튼을 누르자 저장된 값이 LED 하위 4개에 표시되는 것을 확인하였다. 그림 34는 읽기 결과가 LED에 나타난 시점의 사진이며, 스위치 설정부터 쓰기, 읽기까지의 과정은 시연 영상에 기록하였다.

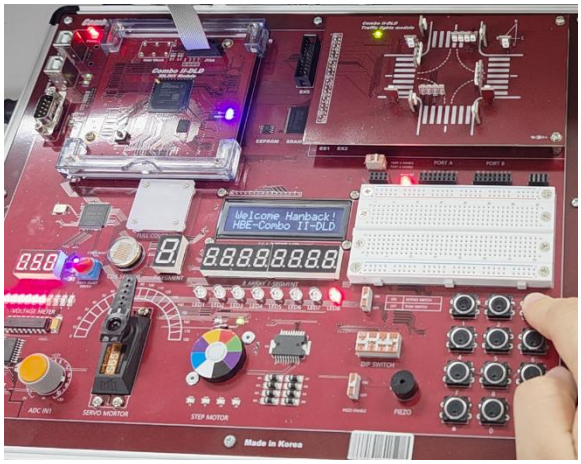


그림 34: 실습2 FPGA 보드 시연 사진

링크: FPGA 실습2 보드 시연 영상

9. FPGA 실습3: DAC 출력

회로 구성

ramp8은 8비트 값을 차례로 증가시킨다. dac_parallel8은 갱신 시점에 선택한 채널과 데이터 값을 DAC 입력에 전달하고 쓰기 신호를 발생시킨다. 최상위 dac은 두 모듈을 연결하고, DAC로 보낸 데이터 dac_d를 led_out에도 연결한다. 따라서 LED 출력과 DAC 입력의 디지털 값이 같아야 한다.

Vivado 실험사진

```
run all
PASS DAC: 260 writes, wraparound, channel, data hold, reset
$finish called at time : 10436 ns : File "D:/Engineering/Verilog_Lab/ece2-lab6/fpga/dac/sim/tb.sv" Line 78
```

그림 35: 실습3 Vivado 시뮬레이션 PASS

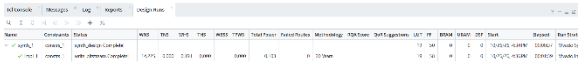


그림 36: 실습3 합성 및 구현 완료 결과

Summary

Resource	Utilization	Available	Utilization %
LUT	19	48000	0.04
FF	58	96000	0.06
IO	23	338	6.80

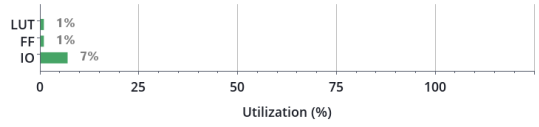


그림 37: 실습3 FPGA 자원 사용량

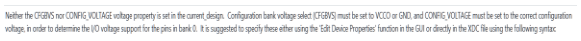


그림 38: 실습3 DRC 검사 결과

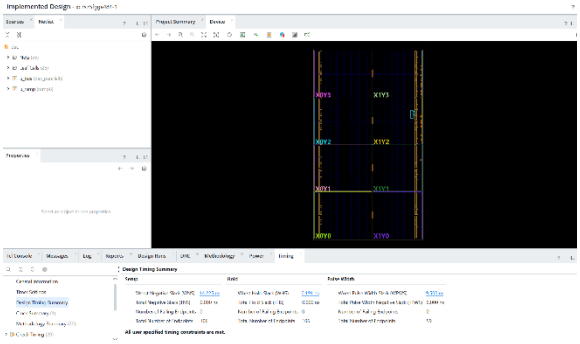


그림 39: 실습3 타이밍 분석 결과



그림 40: 실습3 Bit Hash 확인

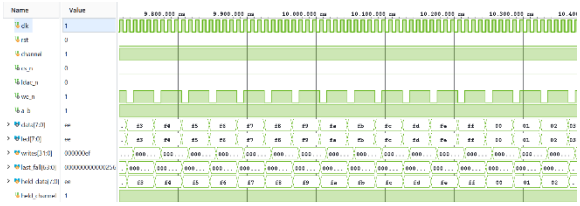


그림 41: 실습3 입력 조합에 따른 출력 파형

Vivado 시뮬레이션 및 파형 비교

그림 41에서 data[7:0]과 led[7:0]은 같은 값으로 변한다. 파형에는 F3 → … → FF → 00 → 01의 순서가 표시되어, 8비트 값이 FF 다음에 00으로 순환하는 것을 확인할 수 있다. we_n의 쓰기 펄스와 채널 선택 신호도 파형에 함께 나타난다.

FPGA 보드 시연

그림 42에서는 DAC 부분, 전압 표시부와 LED가 포함된 보드 시연 상태가 나타난다. 보드 출력을 통한 LED 값의 지속적인 변화는 시연영상으로 확인할 수 있다.

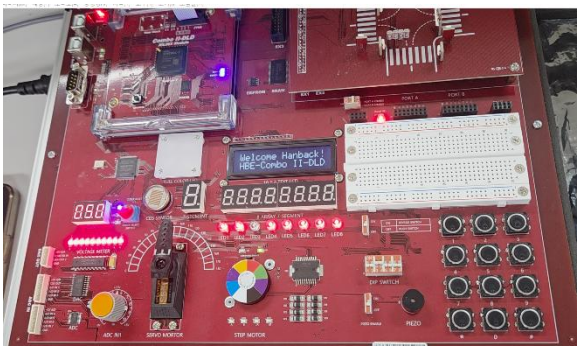


그림 42: 실습3 FPGA 보드 시연 사진

링크: FPGA 실습3 보드 시연 영상

10. 문제 발생 및 해결 과정

UART 시연 중 USB 포트 연결 문제

STM32와 FPGA를 함께 사용하면서 FPGA 프로그램용 연결과 시리얼 모니터용 연결이 필요했지만, 노트북에 남는 포트가 부족하여 USB-C 변환 허브를 사용하였다. 처음에는 이 연결 상태에서 포트 사용이 원활하지 않아 통신 결과를 확인하기 어려웠다.

이를 해결하기 위해 먼저 FPGA에 비트스트림을 다운로드하였다. 이후 FPGA 보드의 전원을 유지한 상태에서 프로그램 다운로드에 사용한 연결을 분리하고, 확보한 포트를 시리얼 모니터 연결에 사용하였다. 연결을 조정한 뒤에는 출력 기록을 확인할 수 있었다. 이는 전원이 유지되는 동안 FPGA에 프로그램한 동작이 유지되는 성질을 이용한 것이다.

11. 예상 값 · Vivado · 보드 결과 비교

실험	예상 및 판정 기준	실제보드 확인
과제5	입력 문자와 해당 바이트의 16진수 출력	Z, A, a 값 출력
과제6	STM32가 보낸 문자가 FPGA Echo 출력에 나타남	MCU 송신 로그와 출력기록
과제7	0 끄기, 1 켜기, 2 반전	LED 보드 시연
실습1	설정에 따라 서보 PWM 펄스폭 변경	서보모터 시연
실습2	SRAM 주소별 쓰기 및 읽기	SRAM 보드 표시 시연
실습3	DAC 입력 데이터 증가 및 디지털 값 순환	DAC 보드 출력 시연

STM32 빌드는 프로그램이 생성되었다는 결과이고, FPGA 시뮬레이션은 테스트벤치에서 지정한 디지털 조건을 확인한 결과다. 실제 보드의 연속 동작은 각 시연 영상과 함께 비교하였다.

12. Bitstream 해시 표

실험	Bitstream	SHA-256
과제6	Lab6_uart_echo.bit	CDE9A8A...
과제7	Lab6_uart_led.bit	0B03D01...
Servo	Lab6_servo.bit	8231FD8...
SRAM	Lab6_sram.bit	EA3ABA6...
DAC	Lab6_dac.bit	885CE8E...

링크: 비트스트림 파일

13. 결론

이번 실험에서는 PC에서 보낸 문자가 STM32를 거쳐 FPGA까지 전달되는 과정을 직접 확인하였다. 과제 5에서는 입력 문자가 화면에 보이는 글자일 뿐 아니라 서로 다른 바이트 값으로 처리된다는 점을 확인했고, 과제 6·7에서는 STM32의 송신 결과와 FPGA의 출력·LED 반응을 각각 확인해야 통신 경로 전체의 동작을 판단할 수 있음을 알게 되었다.

서보모터 실습에서는 PWM을 바꾸면 모터의 속도가 달라지는 것으로만 생각했는데, 이번 회로에서는 펄스 폭으로 목표 위치를 정한다는 점이 인상적이었다. 4번과 6번 버튼으로 모터를 양쪽으로 움직이고 5번 리셋 버튼으로 중간 위치에 돌아오게 하면서 파형의 변화가 실제 움직임으로 이어지는 것을 볼 수 있었다. SRAM 실습에서는 스위치 값을 저장한 뒤 다시 읽어 LED로 확인하면서 쓰기와 읽기 때 데이터 버스의 역할이 달라진다는 점을 이해했다. DAC 실습에서는 디지털 값이 증가하고 순환하는 파형을 보드 출력과 연결해 살펴보았다.

시연 중에는 노트북 포트가 부족해 허브를 사용했지만 처음 연결 상태에서는 통신 확인이 잘되지 않았다. FPGA에 프로그램을 먼저 넣고 연결을 조정한 뒤 동작을 확인하면서, 코드와 시뮬레이션이 맞더라도 실제 실험에서는 전원·케이블·포트 연결까지 확인해야 한다는 것을 느꼈다.

참고 문헌

[1] Cypress Semiconductor Corporation, *CY7C1061GN/CY7C10612GN: 16-Mbit (1M Words × 16 Bit) Static RAM*, Datasheet, Document No. 001-93680, Rev. *C, 2016, pp. 1–2.

https://www.infineon.com/assets/row/public/documents/10/49/infineon-cy7c1061gn-cy7c10612gn-16-mbit-1m-words-16-bit-static-ram-datasheet-en.pdf